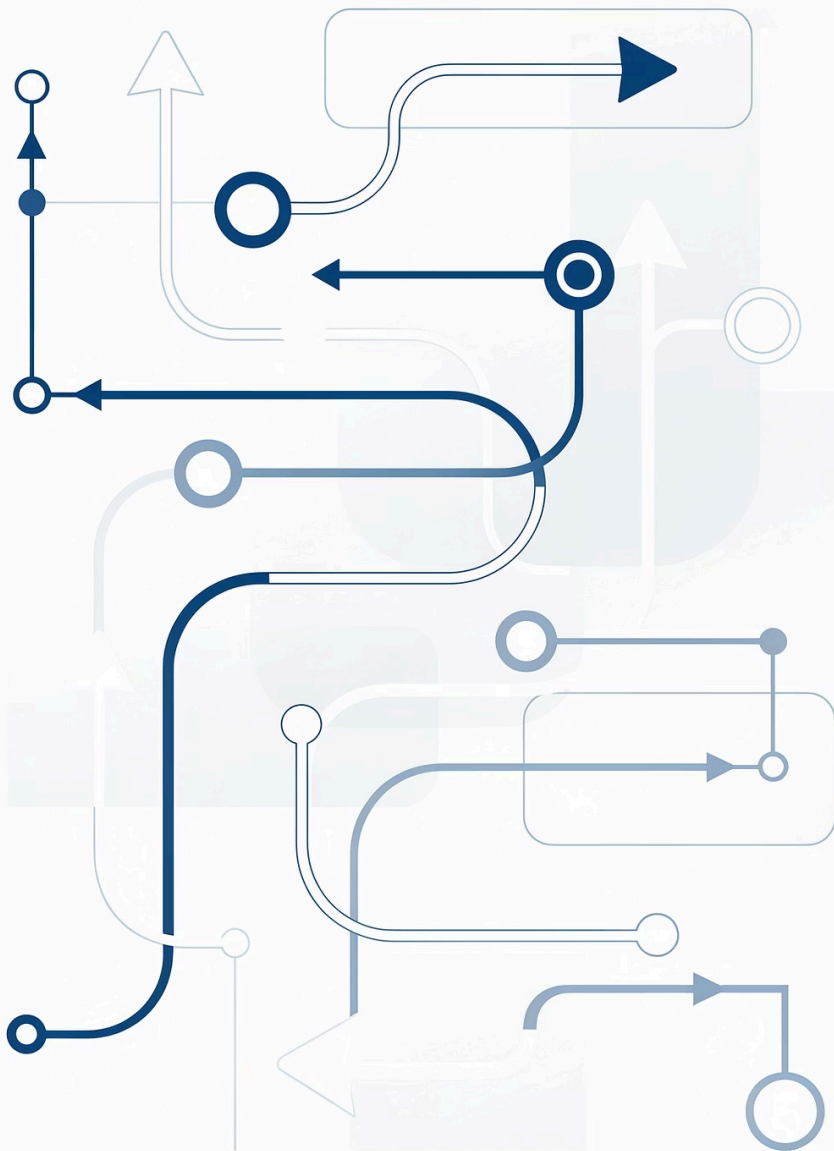




Patrón de Diseño Command

También conocido como: **Comando**, **Orden**, **Action**, **Transaction**

Un patrón de comportamiento que transforma solicitudes en objetos independientes, habilitando parametrización, colas de ejecución y operaciones reversibles.

Índice de Contenidos

Un recorrido guiado a través de los conceptos clave y la implementación del Patrón Command.

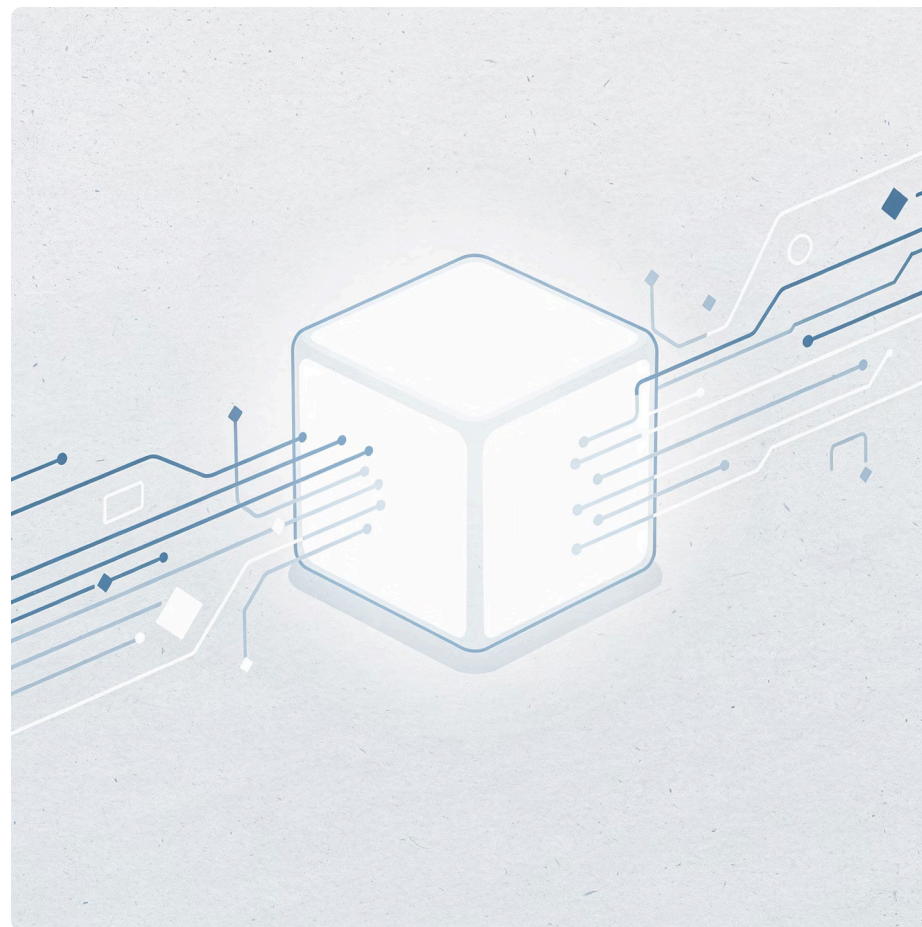
01	02	03
¿Qué es el Patrón Command?	El Problema	La Solución
04	05	06
Analogía en el Mundo Real	Estructura del Patrón	Pseudocódigo: Editor de Texto
07	08	09
Aplicabilidad del Patrón	Pros y Contras	Relaciones con Otros Patrones

¿Qué es el Patrón Command?

Command es un **patrón de diseño de comportamiento** que convierte una solicitud en un objeto independiente que contiene toda la información sobre dicha solicitud.

Esta transformación te permite:

- **Parametrizar** métodos con diferentes solicitudes
- **Retrasar o poner en cola** la ejecución de una solicitud
- **Soportar operaciones** que no se pueden realizar (deshacer/rehacer)



El Problema

Imagina que estás trabajando en una nueva **aplicación de edición de texto**. Tu tarea actual consiste en crear una barra de herramientas con botones para varias operaciones del editor. Creaste una clase `Botón` muy limpia que puede utilizarse tanto para la barra de herramientas como para botones genéricos en diversos diálogos.



El Dilema de los Botones

Todos los botones de la aplicación provienen de la misma clase. Aunque se parecen, se supone que hacen cosas diferentes. **¿Dónde pondrías el código para los gestores de clics?**

Solución Ingenua

Crear cientos de subclases para cada lugar donde se utilice el botón, conteniendo el código que se ejecuta al hacer clic.

Problema Resultante

Enorme cantidad de subclases con riesgo de descomponer el código cada vez que se modifica la clase base `Botón`.

Dependencia Peligrosa

El código GUI depende torpemente del volátil código de la lógica de negocio.

La Parte Más Desagradable

Duplicación de Código

Algunas operaciones como **copiar/pegar texto** deben ser invocadas desde varios lugares:

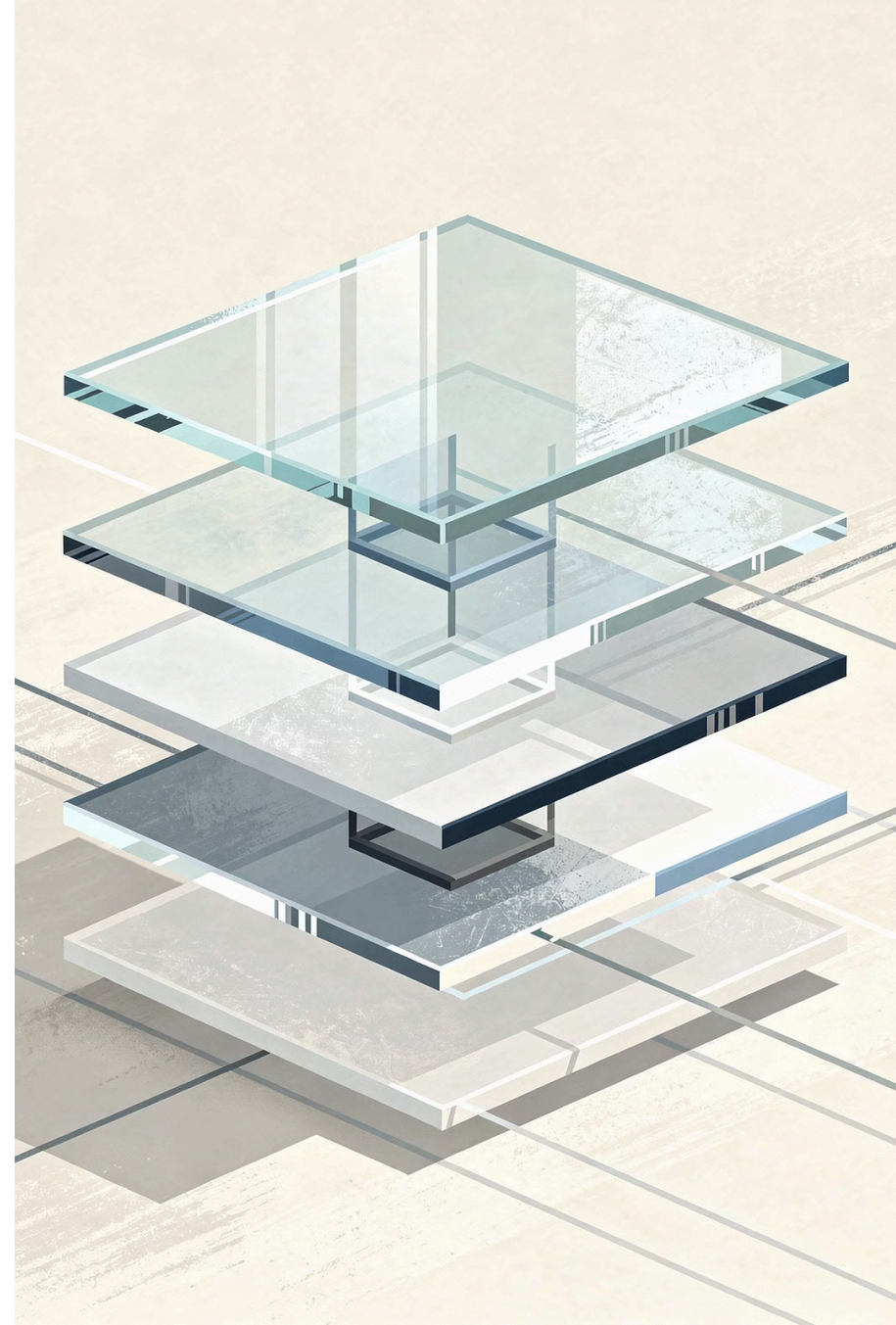
- Botón "Copiar" en la barra de herramientas
- Menú contextual
- Atajo de teclado `Ctrl+C`

Inicialmente, con solo la barra de herramientas, tener el código para copiar texto dentro de la subclase `BotónCopiar` estaba bien.

Sin embargo, cuando implementas **menús contextuales, atajos y otros elementos**, debes duplicar el código de la operación en muchas clases, o bien hacer menús dependientes de los botones — una opción aún peor.

La Solución

El buen diseño de software se basa en el **principio de separación de responsabilidades**, lo que suele resultar en la división de la aplicación en capas: una capa para la **interfaz gráfica (GUI)** y otra para la **lógica de negocio**.



Separación GUI y Lógica de Negocio

1

Capa GUI

Responsable de representar la imagen en pantalla, capturar entradas y mostrar resultados de lo que el usuario y la aplicación están haciendo.

2

Capa de Lógica de Negocio

Se encarga de lo importante: calcular, procesar y ejecutar las operaciones reales. La GUI delega el trabajo a esta capa subyacente.

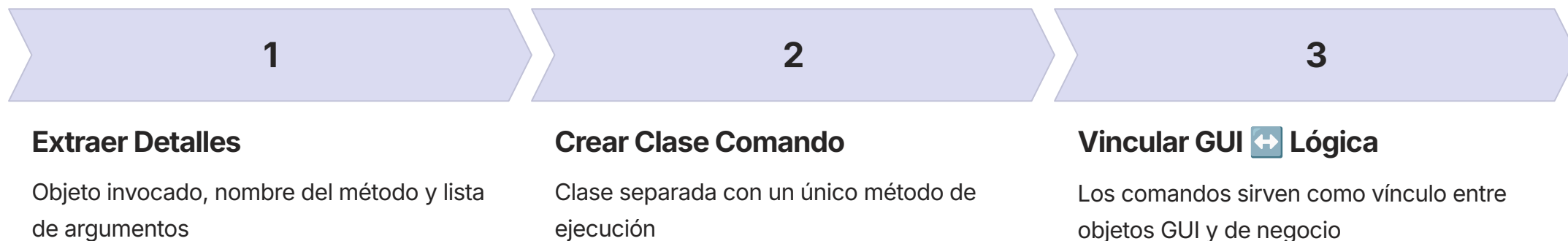
3

Comunicación

Un objeto GUI invoca un método de un objeto de lógica de negocio, pasándole argumentos. Un objeto envía a otro una **solicitud**.

La Propuesta del Patrón Command

El patrón Command sugiere que los objetos GUI **no envíen solicitudes directamente**. En su lugar, debes extraer todos los detalles de la solicitud y ponerlos dentro de una **clase comando separada** con un único método que activa la solicitud.



De ahora en adelante, el objeto GUI no tiene que conocer qué objeto de la lógica de negocio recibirá la solicitud ni cómo la procesará. El objeto GUI simplemente **activa el comando**, que gestiona todos los detalles.

Interfaz Común y Desacoplamiento

Interfaz Unificada

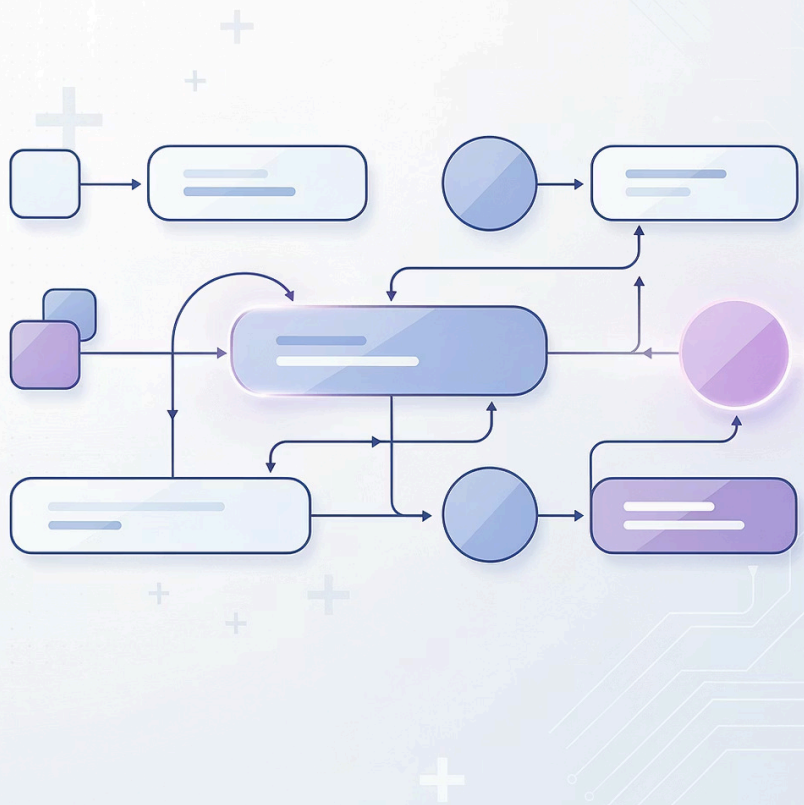
Todos los comandos implementan la **misma interfaz**, normalmente con un único método de ejecución sin parámetros. Esto permite utilizar varios comandos con el mismo emisor sin acoplarlo a clases concretas.

Puedes **cambiar objetos de comando** vinculados al emisor, cambiando efectivamente su comportamiento durante el tiempo de ejecución.

¿Y los Parámetros?

Si el método de ejecución no acepta parámetros, ¿cómo pasamos los detalles de la solicitud al receptor?

- ❑ El comando debe estar **preconfigurado** con la información necesaria o ser capaz de conseguirla por su cuenta.



Resultado: Eliminando la Duplicación

Tras aplicar el patrón Command, ya no necesitamos todas esas subclases de botón. Basta con colocar un **único campo** dentro de la clase base Botón que almacene una referencia a un objeto de comando y haga que el botón ejecute ese comando en un clic.

Otros elementos GUI — menús, atajos o diálogos enteros — se implementan del mismo modo. Los elementos relacionados con las mismas operaciones se vinculan a los **mismos comandos**, evitando cualquier duplicación de código. Los comandos se convierten en una **conveniente capa intermedia** que reduce el acoplamiento entre GUI y lógica de negocio.

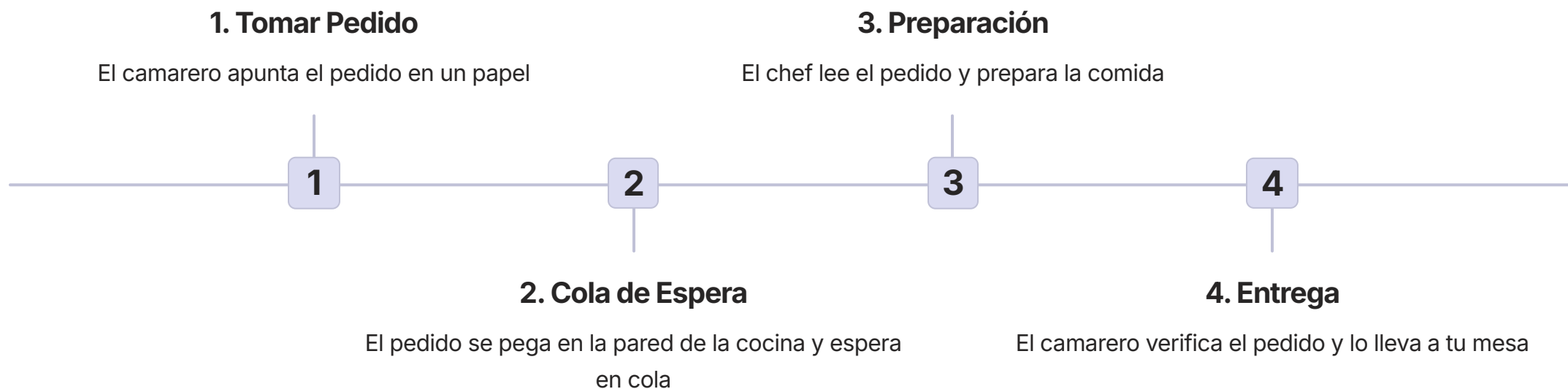


🏠 CAPÍTULO 3

Analogía en el Mundo Real

Tras un largo paseo por la ciudad, entras en un buen restaurante y te sientas a una mesa junto a la ventana. Un amable camarero toma tu pedido rápidamente, apuntándolo en un papel.

El Pedido como Comando

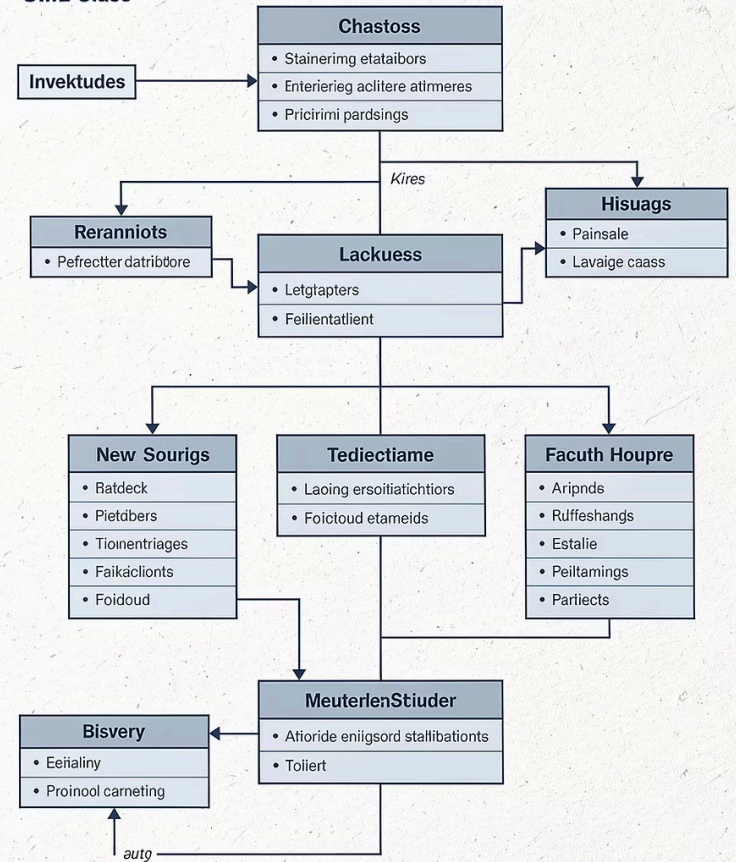


- 📄 El pedido en papel hace la función de un **comando**. Contiene toda la información relevante necesaria para preparar la comida. Permite al chef empezar a cocinar de inmediato, en lugar de tener que aclarar los detalles directamente contigo.

Estructura del Patrón

La estructura del patrón Command se compone de **cinco participantes clave** que colaboran para desacoplar al emisor de la solicitud del receptor que la ejecuta.

UML Class



Los 5 Participantes

1 Emisora (Invocadora)

Responsable de inicializar las solicitudes. Tiene un campo para almacenar una referencia a un objeto de comando. Activa el comando en lugar de enviar la solicitud directamente al receptor. No crea el objeto de comando; lo obtiene del cliente a través del constructor.

2 Interfaz Comando

Normalmente declara un **único método** para ejecutar el comando.

3 Comandos Concretos

Implementan varios tipos de solicitudes. No realizan el trabajo por su cuenta, sino que pasan la llamada a objetos de la lógica de negocio. Los parámetros necesarios se declaran como campos en el comando concreto, inicializados a través del constructor.

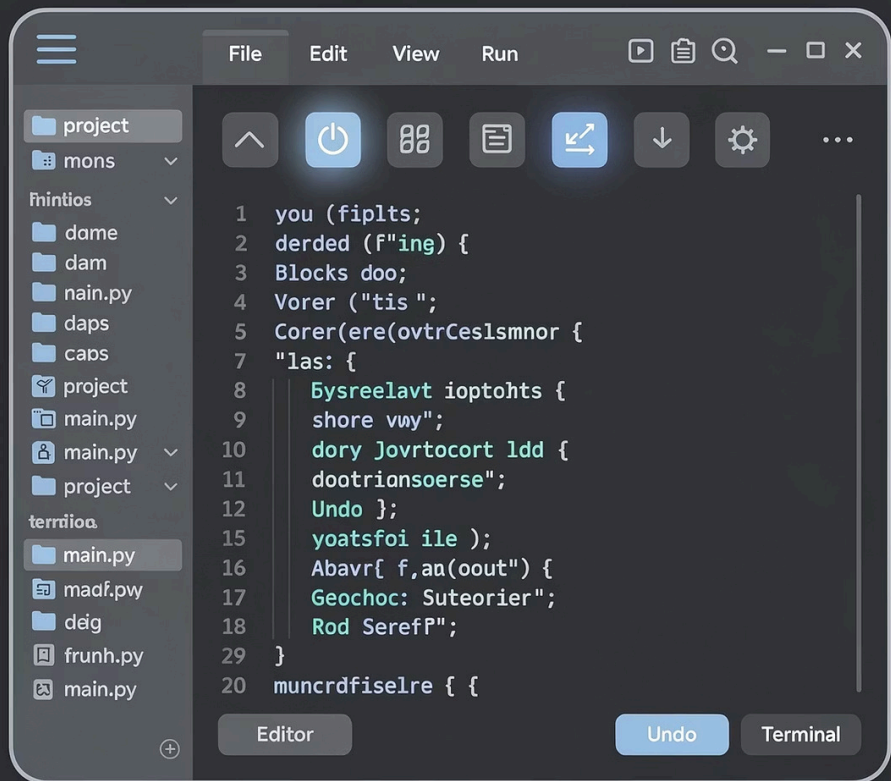
Participantes: Receptor y Cliente

4. Clase Receptora

Contiene cierta **lógica de negocio**. Casi cualquier objeto puede actuar como receptor. La mayoría de los comandos solo gestiona los detalles sobre cómo se pasa una solicitud al receptor, mientras que el propio receptor hace el **trabajo real**.

5. El Cliente

Crea y configura los objetos de comando concretos. Debe pasar **todos los parámetros** de la solicitud, incluyendo una instancia del receptor, dentro del constructor del comando. Después, el comando resultante puede asociarse con uno o varios emisores.



CAPÍTULO 5

Pseudocódigo: Editor de Texto

En este ejemplo, el patrón Command ayuda a **rastrear el historial de operaciones ejecutadas** y hace posible revertir una operación si es necesario. Los comandos que cambian el estado del editor realizan una **copia de seguridad** antes de ejecutar la operación asociada.

Clase Base Command

La clase base define la interfaz común a todos los comandos concretos, incluyendo métodos para copia de seguridad y deshacer:

```
// La clase base comando define la interfaz común a todos los
// comandos concretos.
abstract class Command is
  protected field app: Application
  protected field editor: Editor
  protected field backup: text

  constructor Command(app: Application, editor: Editor) is
    this.app = app
    this.editor = editor

  // Realiza una copia de seguridad del estado del editor.
  method saveBackup() is
    backup = editor.text

  // Restaura el estado del editor.
  method undo() is
    editor.text = backup

  // El método de ejecución se declara abstracto para forzar a
  // todos los comandos concretos a proporcionar sus propias
  // implementaciones. El método debe devolver verdadero o
  // falso dependiendo de si el comando cambia el estado del
  // editor.
  abstract method execute()
```

Comandos Concretos: Copy, Cut, Paste

Cada comando concreto implementa una operación específica del editor:

```
// Los comandos concretos van aquí.  
class CopyCommand extends Command is  
  // El comando copiar no se guarda en el historial ya que no  
  // cambia el estado del editor.  
  method execute() is  
    app.clipboard = editor.getSelection()  
    return false
```

```
class CutCommand extends Command is  
  // El comando cortar cambia el estado del editor, por lo  
  // que debe guardarse en el historial.  
  method execute() is  
    saveBackup()  
    app.clipboard = editor.getSelection()  
    editor.deleteSelection()  
    return true
```

```
class PasteCommand extends Command is  
  method execute() is  
    saveBackup()  
    editor.replaceSelection(app.clipboard)  
    return true
```

```
// La operación deshacer también es un comando.  
class UndoCommand extends Command is  
  method execute() is  
    app.undo()  
    return false
```

Historial de Comandos y Editor

El historial global es una pila, y la clase Editor actúa como receptor:

```
// El historial global de comandos tan solo es una pila.
class CommandHistory is
  private field history: array of Command

  // El último dentro...
  method push(c: Command) is
    // Empuja el comando al final de la matriz del
    // historial.

    // ...el primero fuera.
    method pop():Command is
      // Obtiene el comando más reciente del historial.

  // La clase editora tiene operaciones reales de edición de
  // texto. Juega el papel de un receptor: todos los comandos
  // acaban delegando la ejecución a los métodos del editor.
  class Editor is
    field text: string

    method getSelection() is
      // Devuelve el texto seleccionado.

    method deleteSelection() is
      // Borra el texto seleccionado.

    method replaceSelection(text) is
      // Inserta los contenidos del portapapeles en la
      // posición actual.
```


Clase Application (Emisor)

La clase Application establece relaciones entre objetos y actúa como emisor:

```
class Application is
  field clipboard: string
  field editors: array of Editors
  field activeEditor: Editor
  field history: CommandHistory

  method createUI() is
    copy = function() { executeCommand(
      new CopyCommand(this, activeEditor)) }
    copyButton.setCommand(copy)
    shortcuts.onKeyPress("Ctrl+C", copy)

    cut = function() { executeCommand(
      new CutCommand(this, activeEditor)) }
    cutButton.setCommand(cut)
    shortcuts.onKeyPress("Ctrl+X", cut)

    paste = function() { executeCommand(
      new PasteCommand(this, activeEditor)) }
    pasteButton.setCommand(paste)
    shortcuts.onKeyPress("Ctrl+V", paste)

    undo = function() { executeCommand(
      new UndoCommand(this, activeEditor)) }
    undoButton.setCommand(undo)
    shortcuts.onKeyPress("Ctrl+Z", undo)

  method executeCommand(command) is
    if (command.execute())
      history.push(command)

  method undo() is
    command = history.pop()
    if (command != null)
      command.undo()
```

Aplicabilidad del Patrón



Parametrizar Objetos con Operaciones

Convierte una llamada a un método en un objeto autónomo. Puedes pasar comandos como argumentos, almacenarlos dentro de otros objetos y cambiar comandos vinculados durante el tiempo de ejecución. Ejemplo: un menú contextual configurable.



Poner en Cola o Programar Ejecución

Un comando se puede **serializar** (convertir en cadena para archivo o base de datos). Puedes retardar, programar, poner en cola, registrar o enviar comandos por la red.



Implementar Operaciones Reversibles

El patrón Command es quizá la forma más popular de implementar **deshacer/rehacer**. El historial es una pila de objetos de comando ejecutados junto a copias de seguridad del estado.

Desventajas del Deshacer/Rehacer

Estado Privado

No es fácil guardar el estado de una aplicación porque parte puede ser privada. Este problema puede mitigarse con el patrón **Memento**.

Consumo de Memoria

Las copias de seguridad de estado pueden consumir mucha RAM. Como alternativa, el comando puede realizar la **operación inversa**, aunque puede resultar difícil o imposible de implementar.

Cómo Implementarlo

01

Declarar la Interfaz

Declara la interfaz de comando con un único método de ejecución.

02

Extraer Solicitudes

Ponlas dentro de clases concretas de comando que implementen la interfaz. Cada clase debe tener campos para argumentos y referencias al receptor, inicializados vía constructor.

03

Identificar Emisoras

Añade campos para almacenar comandos. Las emisoras se comunican solo a través de la interfaz de comando. Normalmente obtienen los comandos del código cliente.

04

Cambiar Emisoras

Ejecutan el comando en lugar de enviar directamente una solicitud al receptor.

05

Inicializar en Orden

El cliente debe: crear receptores → crear comandos y asociarlos con receptores → crear emisores y asociarlos con comandos específicos.

Pros y Contras

✅ Ventajas

- **Principio de responsabilidad única:** Desacoplas las clases que invocan operaciones de las que las realizan
- **Principio abierto/cerrado:** Introduces nuevos comandos sin descomponer el código existente
- Puedes implementar **deshacer/rehacer**
- Puedes implementar la **ejecución diferida** de operaciones
- Puedes ensamblar un grupo de **comandos simples** para crear uno complejo

❌ Desventajas

- ❏ El código puede complicarse, ya que estás introduciendo una **nueva capa** entre emisores y receptores.

Relaciones con Otros Patrones

Chain of Responsibility

Pasa solicitudes secuencialmente por una cadena dinámica de receptores. Los manejadores pueden implementarse como Comandos.

Mediator / Observer

Mediator fuerza comunicación indirecta vía objeto mediador. Observer permite suscripción dinámica a solicitudes.

Memento

Úsalo junto con Command para "deshacer": los comandos operan sobre el objeto destino y los mementos guardan el estado previo.

Strategy

Ambos parametrizan objetos con acciones, pero Command convierte operaciones en objetos (aplazar, encolar), mientras Strategy intercambia algoritmos.

Prototype / Visitor

Prototype ayuda a guardar copias de Comandos en historial. Visitor es una versión potente de Command que ejecuta operaciones sobre objetos de distintas clases.